204252 Data Structures and Analysis

Chapter 11: Graphs (Part 2) - Disjoint Sets

Lecturer

August 31, 2026

Computer Science Department

Outline

The Disjoint Set Union (DSU) ADT

Smart Union Strategies

Path Compression

Application: Dynamic Maze Generation

The Disjoint Set Union (DSU) ADT

Equivalence Relations & Partitions

- An **equivalence relation** R over a set S is a relation that is:
  1. **Reflexive**: aRa for all $a \in S$.
  2. **Symmetric**: aRb if and only if bRa.
  3. **Transitive**: if aRb and bRc, then aRc.
- An equivalence relation partitions S into disjoint **equivalence classes**:
  - Every element $a \in S$ belongs to exactly one equivalence class.
  - Two elements are in the same class if and only if they are related.
- **Goal of DSU**: Maintain these partitions dynamically under merges.

The Disjoint Set Abstract Data Type

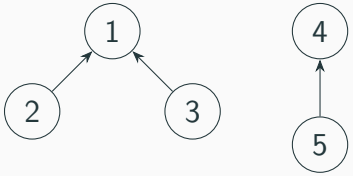
The Disjoint Set (or Union-Find) ADT supports two primary operations:

1. **Find(x)**: Returns the name of the set (equivalence class) containing x. Usually, this is represented by a unique **representative element** of the set.
2. **Union(root1, root2)**: Merges the two sets containing elements in equivalence classes `root1` and `root2` into a single equivalence class.

Important Constraint
The sets are strictly **disjoint** (no element belongs to more than one set).

Tree Representation of Disjoint Sets

- We represent each set as a **tree**, where each node points to its parent.
- The root of each tree is the representative of the set.
- The root node's parent pointer points to itself or has a negative value.



Parent Array Representation:

Index i	1	2	3	4	5
Parent $s[i]$	-1	1	1	-1	4

Negative values indicate roots (value represents height or size).

Simple Union and Find Implementations

Using a vector s initialized to −1:

```
1  // Construct Disjoint Sets
2  DisjSets::DisjSets(int numElements)
3      : s(numElements, -1) {}
4
5  // Simple Find
6  int DisjSets::find(int x) const {
7      if (s[x] < 0)
8          return x;
9      else
10         return find(s[x]);
11 }
```

```
1  // Simple Union: root2 points to root1
2  void DisjSets::unionSets(int root1, int root2) {
3      s[root2] = root1;
4  }
```

Worst-case: A tree of height $N - 1$ can be created. Find operations can take $\mathcal{O}(N)$ time.

Smart Union Strategies

Union-by-Size

- **Strategy**: Always make the smaller tree a subtree of the larger tree.
- **Implementation**: Store the negative size of the tree at the root. For example, if a tree has size 3, the root stores −3.
- **Result**: The depth of any node is at most $\lfloor \log_2 N \rfloor$.

```
1  void DisjSets::unionBySize(int root1, int root2) {
2      if (s[root2] < s[root1]) { // root2 has more elements (more negative)
3          s[root2] += s[root1];  // Update size of root2
4          s[root1] = root2;      // Make root2 parent of root1
5      } else {
6          s[root1] += s[root2];  // Update size of root1
7          s[root2] = root1;      // Make root1 parent of root2
8      }
9  }
```

Union-by-Height / Rank

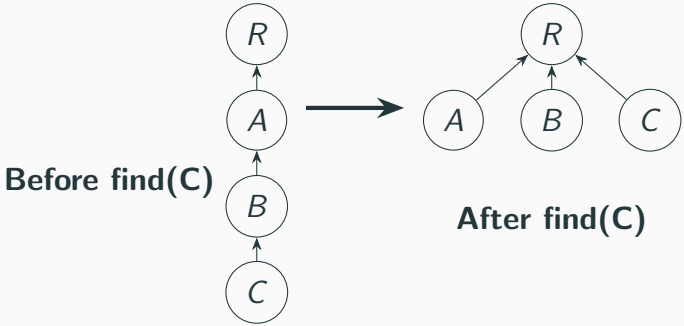
- **Strategy**: Always make the shallower tree a subtree of the deeper tree.
- **Implementation**: Store the negative height at the root. Initially −1 (height 0). If heights are equal, decrement the new root's value (increasing height by 1).
- **Worst-case depth**: $\mathcal{O}(\log N)$.

```
1  void DisjSets::unionByHeight(int root1, int root2) {
2      if (s[root2] < s[root1]) { // root2 is deeper (more negative)
3          s[root1] = root2;
4      } else {
5          if (s[root1] == s[root2]) {
6              s[root1]--;        // heights equal, increase height of root1
7          }
8          s[root2] = root1;
9      }
10 }
```

Path Compression

Path Compression

- Smart union guarantees $\mathcal{O}(\log N)$ height, but we can do even better during find operations.
- **Path Compression**: During a `find(x)`, make every node along the path from x to the root point directly to the root.



Find with Path Compression C++

```
int DisjSets::find(int x) {
    if (s[x] < 0) {
        return x;
    } else {
        return s[x] = find(s[x]); // parent set directly to root of find
    }
}
```

¿ [!NOTE] ¿ This is a very clean recursive implementation. It traverses up to find the root, and as the recursion unwinds, it assigns the parent of every visited node directly to that root.

Asymptotic Complexity Analysis

- When we combine **Union-by-Rank** and **Path Compression**, the running time is almost linear!
- Specifically, any sequence of M finds and $N - 1$ unions takes:

$$\mathcal{O}(M\alpha(M, N))$$

- $\alpha(M, N)$ is the **Inverse Ackermann Function**.
- This function grows so incredibly slowly that for all practical values (e.g. $N < 2^{65536}$), $\alpha(M, N) \leq 4$.
- Effectively, the amortized cost per operation is $\mathcal{O}(1)$.

Application: Dynamic Maze Generation

Dynamic Maze Generation Algorithm

Disjoint Sets can be used to generate a random maze:

1. Start with walls everywhere (grid representation).
2. Initialize a Disjoint Sets structure where each grid cell is in its own set.
3. While the number of disjoint sets is greater than 1:
   - 3.1 Choose a wall randomly that separates cell u and cell v.
   - 3.2 Perform `find(u)` and `find(v)`.
   - 3.3 If they are in different sets: knock down the wall, and perform `unionSets(find(u), find(v))`.
   - 3.4 If they are in the same set: do nothing (knocking down the wall would create a loop/cycle).

Maze Generation (Initial Grid (All Walls Up))



Step Description:
All cells are in separate disjoint sets. All walls are standing.

Disjoint Sets (Partitions):
$\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}$

Maze Generation (Step 1: Choose Wall between 0 and 1)



Step Description:

Selected wall between cell **0** and cell **1**.

`find(0)` $= 0$

`find(1)` $= 1$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

{0, 1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}

Maze Generation (Step 2: Choose Wall between 4 and 7)



Step Description:

Selected wall between cell **4** and cell **7**.

`find(4)` $= 4$

`find(7)` $= 7$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1\}, \{2\}, \{3\}, \{4, 7\}, \{5\}, \{6\}, \{8\}$

Maze Generation (Step 3: Choose Wall between 1 and 2)



Step Description:

Selected wall between cell **1** and cell **2**.

`find(1)` $= 1$

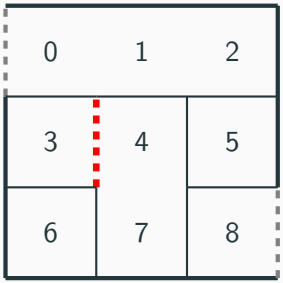
`find(2)` $= 7$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1, 2\}$, $\{3\}$, $\{4, 7\}$, $\{5\}$, $\{6\}$, $\{8\}$

Maze Generation (Step 4: Choose Wall between 3 and 4)



Step Description:

Selected wall between cell **3** and cell **4**.

`find(3)` $= 3$

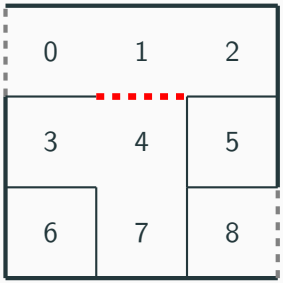
`find(4)` $= 7$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1, 2\}$, $\{3, 4, 7\}$, $\{5\}$, $\{6\}$, $\{8\}$

Maze Generation (Step 5: Choose Wall between 1 and 4)



Step Description:

Selected wall between cell **1** and cell **4**.

`find(1)` $= 1$

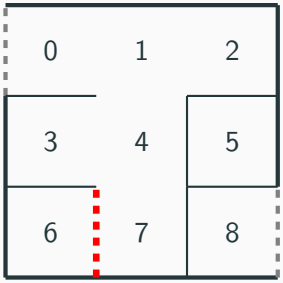
`find(4)` $= 0$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1, 2, 3, 4, 7\}$, $\{5\}$, $\{6\}$, $\{8\}$

Maze Generation (Step 6: Choose Wall between 6 and 7)



Step Description:

Selected wall between cell **6** and cell **7**.

`find(6)` $= 0$

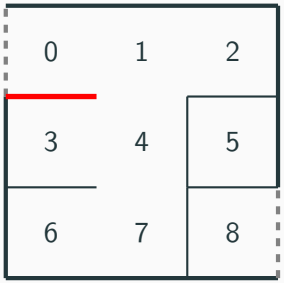
`find(7)` $= 0$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1, 2, 3, 4, 6, 7\}$, $\{5\}$, $\{8\}$

Maze Generation (Step 7: Choose Wall between 0 and 3)



Step Description:

Selected wall between cell **0** and cell **3**.

`find(0)` $= 0$

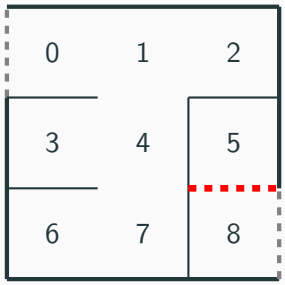
`find(3)` $= 0$

Status: **Rejected (Already Connected!)**

Disjoint Sets (Partitions):

$\{0, 1, 2, 3, 4, 6, 7\}$, $\{5\}$, $\{8\}$

Maze Generation (Step 8: Choose Wall between 5 and 8)



Step Description:

Selected wall between cell **5** and cell **8**.

`find(5)` $= 0$

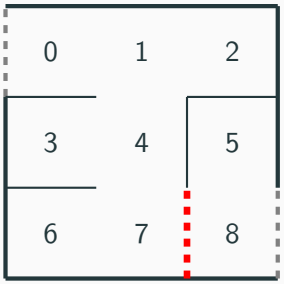
`find(8)` $= 0$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1, 2, 3, 4, 6, 7\}$, $\{5, 8\}$

Maze Generation (Step 9: Choose Wall between 7 and 8)



Step Description:

Selected wall between cell **7** and cell **8**.

`find(7)` $= 0$

`find(8)` $= 0$

Status: **Knocked Down!**

Disjoint Sets (Partitions):

$\{0, 1, 2, 3, 4, 5, 6, 7, 8\}$